Jamie Goodrich-Maclean
1825924

Windows Application Programming Lesson 2
Signals & Slots

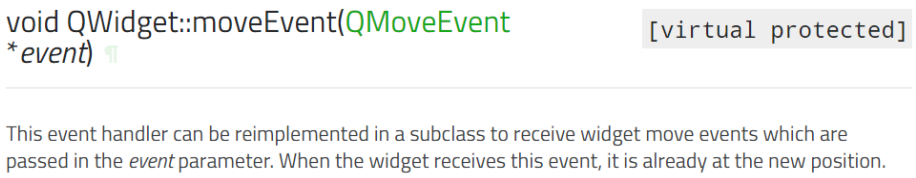
This lesson covers the following topics.

- Signals and slots
- Events

Signals, Slots & Events:

In a typical application a key press of a keyboard or a button press on a mouse can trigger an 'Event'. These event is very similar to hardware interrupts in microcontroller programming. Typically we can re-implement event handlers which are methods that run when their corresponding events occur.

EX1. Look up the documentation associated with the QWidget Object. Locate the Event handlers. Are these event handlers declared with the keywords 'virtual' & 'protected'? What do these two words mean? (Hint: https://en.cppreference.com/w/cpp/language/access)



The protected keyword states that class members can be accessed by the class and subclasses but aren't intended to be used externally. In QWidget classes, event handlers are typically marked as protected to show they're for use mainly with derived classes and not external access.

The virtual keyword allows derived classes to override base class functions. You can add, edit the original base class for implementations such as custom event handling in event handlers.

EX2. Try the following example:

Header file.

```
#ifndef CODEEXAMPLE_H
#define CODEEXAMPLE_H

#include <QMainWindow>
#include <QMessageBox>
#include <QKeyEvent>

class CodeExample : public QWidget
{
    Q_OBJECT

public:
    CodeExample(QWidget *parent = 0);
    ~CodeExample();

protected:
```

```
    void keyPressEvent(QKeyEvent * e);
};

#endif // CODEEXAMPLE_H
```

implementation file:

```
#include "codeexample.h"
CodeExample::CodeExample(QWidget *parent)
    : QWidget(parent)
{
}

CodeExample::~CodeExample()
{

}

void CodeExample::keyPressEvent(QKeyEvent *event)
{
    if (event->key() == Qt::Key_Escape) {
       this->close();
   }
   else
       QMessageBox::information(this,"Event example","You Pressed \"" + event->text()
+ " \".",QMessageBox::Ok,QMessageBox::Ok);
}
```

main.cpp

```
#include "codeexample.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    CodeExample w;
    w.show();

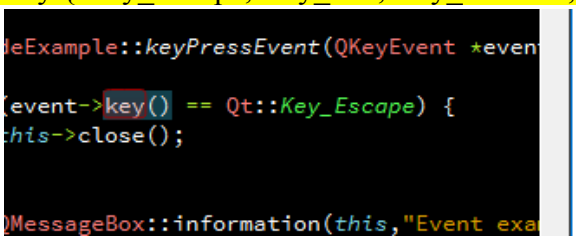
    return a.exec();
}
```

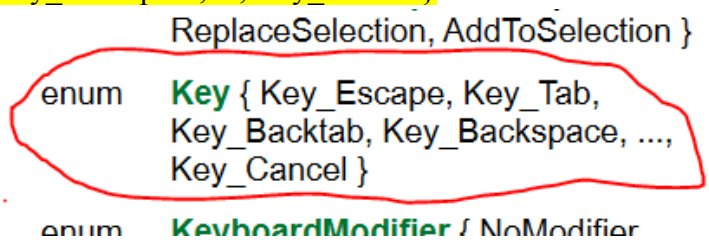
In this example we re-implemented the “KeyPressEvent” handler. This handler passes a pointer to a QkeyEvent object which we can then use to detect which key was pressed on the keyboard. Every time a key is pressed the application jumps to the KeyPressEvent handler. If the key was “Escape”, the program exits else if the key was not escape, the application prints out the key pressed in a message box. The event->Key() method returns a code representing the key pressed. This method can be used to identify all keys..alphanumeric as well as others e.g. shift, backspace, e.t.c. Whereas the event->text() method returns a string representation of the alphanumeric keys only. For non alphanumeric keys, it returns “ “.

Ex3. Look up the QkeyEvent object and the Qt:Key enumeration in the Qt documentation. What is the code and label (enum) for the tab keypress???

code: Qt::Key_Tab

enum:
Key { Key_Escape, Key_Tab, Key_Backtab, Key_Backspace, ..., Key_Cancel }





Ex4. The QmessageBox is a simple dialog that is used to enable communication between the application and the program user. Look up the QmessageBox documentation and explain its usage.

QMessageBox is a class in Qt used for displaying message boxes or dialogs with different types of messages, such as information, warning, critical error, and question messages. It is used for interacting with users and getting their input.

basic message box with user input:

```
int result = QMessageBox::question(nullptr, "Question", "do you like animals?", QMessageBox::Yes | QMessageBox::No);

if (result == QMessageBox::Yes) {
    //clicked Yes
} else {
    //clicked No
}
```

Ex5 Re-implement the QWidget::mouseDoubleClickEvent (QMouseEvent * *event*) Event handler in the above example and use the QmessageBox to indicate if the mouse key double clicked was the left, right or the middle buttons. Include all code in the report (header, implementation & main.cpp).

header:

```
//-------------------------------------------------------------
#ifndef CODEEXAMPLE_H
#define CODEEXAMPLE_H
#include <QMainWindow>
#include <QMessageBox>
#include <QKeyEvent>
class CodeExample : public QWidget
{
  Q_OBJECT
```

```
public:
  CodeExample(QWidget *parent = 0);
  ~CodeExample();
protected:
  void keyPressEvent(QKeyEvent * e);
  void mouseDoubleClickEvent(QMouseEvent *event); //construction for double click event

};
#endif // CODEEXAMPLE_H
//----------------------------------------------------------
implementation:
//----------------------------------------------------------
#include "codeexample.h"

CodeExample::CodeExample(QWidget *parent)
  : QWidget(parent)
{
}

CodeExample::~CodeExample()
{
}

void CodeExample::keyPressEvent(QKeyEvent *event)
{
  if (event->key() == Qt::Key_Escape) {
     this->close();
  } else {
     QMessageBox::information(this, "Event example", "You Pressed \"" + event->text() + " \".",
QMessageBox::Ok, QMessageBox::Ok);
  }
}

void CodeExample::mouseDoubleClickEvent(QMouseEvent *event) //double click event
{
  QString button;
  switch (event->button()) {
  case Qt::LeftButton:
     button = "Left";
     break;
  case Qt::RightButton:
     button = "Right";
     break;
  case Qt::MiddleButton:
     button = "Middle";
     break;
  default:
     button = "Unknown"; //any other double click
  }
```

```
   QMessageBox::information(this, "Mouse Double Click Event", "Mouse button double-clicked: " +
button, QMessageBox::Ok, QMessageBox::Ok); //info box
}
//----------------------------------------------------------
main:
//----------------------------------------------------------
#include "codeexample.h"
#include <QApplication>

int main(int argc, char *argv[])
{
   QApplication a(argc, argv);
   CodeExample w;
   w.show();
   return a.exec();
}
//----------------------------------------------------------
```

Events and Event handlers are the low level event handling mechanism . A higher level event handling mechanism is the signals and slot techniques. When an Event occurs, it emits a signal. The programmer can then associate a signal with a slot method that handles the signals using the “connect()” function call. Custom signals can be defined by the 'emit' Qt keyword. They are not actually implemented. Whereas custom slots are basically normal methods that can be attached to signals.

EX6. Identify the pre-defined signals and the slots of the QPushButton widget from the documentation. https://wiki.qt.io/How_to_Use_QPushButton

Predefined Signals for QPushButton:

clicked(bool checked = false)
pressed()
released()
toggled(bool checked)

Predefined Slots for QPushButton:

click()
animateClick(int msec = 100)
setChecked(bool checked)
setFlat(bool flat)
setDefault(bool isDefault)
handleButton()

EX7. Try the following example:

header file:

```
#ifndef CODEEXAMPLE_H
#define CODEEXAMPLE_H

#include <QMainWindow>
#include <QMessageBox>
#include <QKeyEvent>
#include <QMouseEvent>
#include <QLabel>
#include <QPushButton>
#include <QVBoxLayout>

class CodeExample : public QWidget
{
    Q_OBJECT

public:
    CodeExample(QWidget *parent = 0);
    ~CodeExample();

public slots:
    void buttonPressed(void);

private:
    QVBoxLayout *vbox;
    QLabel *label;
    QPushButton *button;
    int flag;
};

#endif // CODEEXAMPLE_H
```

implementation file

```
#include "codeexample.h"
CodeExample::CodeExample(QWidget *parent)
    : QWidget(parent)
{
    vbox = new QVBoxLayout();
    label = new QLabel("Hello World");
    button = new QPushButton("Hide label");
    flag = 0;
    vbox->addWidget(label);
    vbox->addWidget(button);
    setLayout(vbox);

    connect(button, SIGNAL(clicked()), this, SLOT(buttonPressed()));
}

CodeExample::~CodeExample()
{

}
```

```
void CodeExample::buttonPressed(){
    if (flag == 0){
            label->hide();
            flag = 1;
            button->setText("Show Label");
    }
    else if (flag == 1){
             label->show();
             flag = 0;
             button->setText("Hide Label");
    }
}
```

main.cpp

```
#include "codeexample.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    CodeExample w;
    w.show();

    return a.exec();
}
```

This is a simple application that consists of a label and a button. Every time the button is pressed it emits a signal 'clicked()'. This signal is already defined as part of the QPushButton object. We create a custom slot method 'CodeExample::ButtonPressed' and use the “connect” function to connect the button's clicked signal to our CodeExample's custom slot:

```
connect(button, SIGNAL(clicked()), this, SLOT(buttonPressed())); //  QT4

connect(button, &QPushButton::clicked, this, &CodeExample::buttonPressed); //  QT5
```

This implies that our slot method will be called every time the button object emits a 'clicked()' signal i.e. every time the button is clicked. The slot method hides the label, and then re-displays it on consecutive button clicks. Pre-defined signals and slots are pre-defined for each object. In addition custom signals and slots can also be created by the programmer. In the previous example we used a custom slot with a pre-defined signal. In the example below we associate a predefined signal for the button object to (clicked()) with the pre-defined slot (close()) with this connection we can close the application by clicking on the button.

```
connect(button1, SIGNAL(clicked()), this, SLOT(close())); //  QT4
connect(button1, &QPushButton::clicked, this, &CodeExample::close);//  QT5
```

Custom signals are declared under a “signals:” identifier in the class declaration (in the header file), whereas custom slots are basically normal methods that can be attached to signals. They are declared in the class declaration (in the header file) under one of the following identifiers; I) “public slots:”, ii) “private slots:” or iii) “protected slots:” depending on the desired degree of protection required. Unlike

signals, slots MUST be implemented. In order to use custom slots and/or signals you MUST have the Q_OBJECT macro declared in your class declaration (header).

Ex 8 create a second button "button1" and set its text field to "Quit" and include it in the above application. Associate its clicked() signal with the QWidget's close() slot. Test the behavior of this second button. Include all code in the report (header, implementation & main.cpp).

header:

```
//-------------------------------------------------------------
#ifndef CODEEXAMPLE_H
#define CODEEXAMPLE_H
#include <QMainWindow>
#include <QMessageBox>
#include <QKeyEvent>
#include <QMouseEvent>
#include <QLabel>
#include <QPushButton>
#include <QVBoxLayout>
class CodeExample : public QWidget
{
  Q_OBJECT
public:
  CodeExample(QWidget *parent = 0);
  ~CodeExample();
public slots:
  void buttonPressed(void);
private:
  QVBoxLayout *vbox;
  QLabel *label;
  QPushButton *button; //hide button
  QPushButton *button1; // quit button
  int flag;
};
#endif // CODEEXAMPLE_H
//-------------------------------------------------------------
```

implementation:

```
//-------------------------------------------------------------
#include "codeexample.h"

CodeExample::CodeExample(QWidget *parent)
  : QWidget(parent)
{
  vbox = new QVBoxLayout();
  label = new QLabel("Hello World");
  button = new QPushButton("Hide Label");
  button1 = new QPushButton("quit"); //quit button
  flag = 0;
  vbox->addWidget(label);
```

```
    vbox->addWidget(button);
    vbox->addWidget(button1); // adds quit button to the layout
    setLayout(vbox);

    connect(button, SIGNAL(clicked()), this, SLOT(buttonPressed()));
    connect(button1, SIGNAL(clicked()), this, SLOT(close())); // connect quit button
}

CodeExample::~CodeExample()
{
}

void CodeExample::buttonPressed()
{
    if (flag == 0) {
        label->hide();
        flag = 1;
        button->setText("Show Label");
    } else if (flag == 1) {
        label->show();
        flag = 0;
        button->setText("Hide Label");
    }
}
```

```
//-------------------------------------------------------
main:
//-------------------------------------------------------
#include "codeexample.h"
#include <QApplication>
int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    CodeExample w;
    w.show();
    return a.exec();
}
//-------------------------------------------------------
```

Ex 9 Modify the behavior of the custom slot “buttonPressed()” such that every time “button” is pressed, it alternates between printing two message e.g. “Hello World” & “Qt is awesome”. Include all code in the report (header, implementation & main.cpp).

Jamie Goodrich-Maclean
1825924

header:

```
//---------------------------------------------------------
#ifndef CODEEXAMPLE_H
#define CODEEXAMPLE_H
#include <QMainWindow>
#include <QMessageBox>
#include <QKeyEvent>
#include <QMouseEvent>
#include <QLabel>
#include <QPushButton>
#include <QVBoxLayout>
class CodeExample : public QWidget
{
  Q_OBJECT
public:
  CodeExample(QWidget *parent = 0);
  ~CodeExample();
public slots:
  void buttonPressed(void);
private:
  QVBoxLayout *vbox;
  QLabel *label;
  QPushButton *button; //hide button
  QPushButton *button1; // quit button
  int flag;
  bool showMessage; // boolean for alternating msg
};
#endif // CODEEXAMPLE_H
//---------------------------------------------------------
```

implementation:

```
//---------------------------------------------------------
#include "codeexample.h"

CodeExample::CodeExample(QWidget *parent)
  : QWidget(parent)
{
  vbox = new QVBoxLayout();
  label = new QLabel("Hello World");
  button = new QPushButton("Hide Label");
  button1 = new QPushButton("quit"); //quit button
  flag = 0;
  showMessage = true; // Initialize message to true
  vbox->addWidget(label);
  vbox->addWidget(button);
  vbox->addWidget(button1); // adds quit button to the layout
  setLayout(vbox);

  connect(button, SIGNAL(clicked()), this, SLOT(buttonPressed()));
  connect(button1, SIGNAL(clicked()), this, SLOT(close())); // connect quit button
```

```
}

CodeExample::~CodeExample()
{
}

void CodeExample::buttonPressed()
{
  if (showMessage) {
     label->setText("bruh"); // text 1
  } else {
     label->setText("wow"); //text2
  }

  showMessage = !showMessage; // toggle msg
}
//----------------------------------------------------
main:
//----------------------------------------------------
#include "codeexample.h"
#include <QApplication>
int main(int argc, char *argv[])
{
  QApplication a(argc, argv);
  CodeExample w;
  w.show();
  return a.exec();
}
//----------------------------------------------------
```

Connected signals and slots can also be disconnected using the following notation:

```
disconnect(button1, SIGNAL(clicked()), this, SLOT(close()));//  QT4
disconnect(button1, &QPushButton::clicked,this,&CodeExample::close);//  QT5
```

Note that you can use the QT4 approach to connect() in QT5 but not the other way. Also in QT5 you must disconnect using the same notation used for connecting i.e. if you connect using the QT4 syntax you must must disconnect using the QT4 syntax. Remember the Q_object macro in the header file? This allows us to create custom defined signals and slots. Make sure that you always have it in your header file !!!!! oke 😀

Custom Signals

In the previous example we looked at how to connect QPushButton's predefined signal 'clicked()' to custom (buttonPressed()) and pre-defined slots (close()). In this example we will demonstrate how to create a custom signal. We will also demonstrate how to pass a parameter from the signal to the slot.

Such parameters are typically required by the slot to interpret the signal's status.

header file

```
#ifndef HELLOGUI_H
#define HELLOGUI_H
#include <QWidget>
#include <QLabel>
#include <QVBoxLayout>
#include <QPushButton>
#include <QMessageBox>
#include <QString>

class helloGui : public QWidget
{
    Q_OBJECT
public:
    helloGui(QWidget *parent = 0);
    ~helloGui();
private slots:
    void changeLabelText(void);// 1 - custom slots
    void dispNumTimes(int);  // 2 - custom slots
signals://added
    void counterInc(int);    //3 - custom signal. Declared only. No need to
                               //implement
private:
    QLabel *label;
    QVBoxLayout *vbox;
    QPushButton *button;
    int flag;
    int count;
};

#endif // HELLOGUI_H
```

implementation file

```
#include "hellogui.h"
helloGui::helloGui(QWidget *parent)
    : QWidget(parent)
{
      QVBoxLayout *vbox = new QVBoxLayout();
      label = new QLabel("hello Qt World");
      button = new QPushButton("Click me!");
      label->setFont(QFont("Times New Roman", 10));
      vbox->addWidget(label);
      vbox->addWidget(button);
      setLayout(vbox);
      flag = 0;
      count = 0;
      connect(button, SIGNAL(clicked()), this, SLOT(changeLabelText()));
 //4 - connect predefined QPushButton clicked() signal to custom
 //helloGui::changeLabelText() slot
      connect(this, SIGNAL(counterInc(int)), this, SLOT(dispNumTimes(int)));

//5 - connect custom counterInc(int) signal to custom changeLabelText() slot.
//Notice how both signal and slot in this case are members of the highGUI
//form/widget. Also note how an integer parameter is passed from the signal to the
//slot.
```

```
}


helloGui::~helloGui()
{
}
void helloGui::changeLabelText(void )
{
    if (flag == 0){
        label->setText("Qt is awesome!!!");
        flag = 1;
    }
    else if(flag == 1){
        label->setText("Hello Qt World!!!");
        flag = 0;
    }
    count++;
    emit counterInc(count); // 6 - When the counter is incremented we emit the
    // counterInc signal and pass the counter value as a parameter
}

// 7-Once the counterInc(int) signal is emitted, the dispNumTimes slot is called
//and given the count value as a parameter. The slot method then prints out the
//count value in a Message box
void helloGui::dispNumTimes(int a){
    QString num = QString::number(a);
    QMessageBox::information(this,"Display count", "The button has been pressed " +
num + " times.",QMessageBox::Ok,QMessageBox::Ok);
}
```

main.cpp

```
#include "hellogui.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    helloGui w;
    w.resize(250,150);
    w.setWindowTitle("Hello GUI");
    w.show();

    return a.exec();
}
```

This example demonstrates a slightly more complicated application of the signal / slot mechanism. First a custom signal “counterInc(int)” of the helloGui widget is used along with a custom slot of the same object. Secondly this signal sends an integer parameter to the slot method, which the method then displays via a message box. This integer is a count value that keeps track of the number of times that the button has been pressed.

When a user clicks the button the following sequence takes place:

The predefined 'clicked()' signal is emitted by the QPushButton object 'button'.

This in turn causes code execution to jump to void helloGui::changeLabelText(void) slot method.

In this slot method, the text of the Qlabel object 'label' is modified and the 'count' variable is incremented. The custom void counterInc(int) signal is emitted with the count variable as its parameter.

This in turn causes code execution to jump to void helloGui::dispNumTimes(int a) slot method. The integer parameter count is copied from the signal into the 'a' slot parameter.

The integer parameter is converted to a Qstring using the following instruction:

```
QString num = QString::number(a);
```

This string value is then displayed via a message box.

EX10 In the previous example we saw how we can convert an integer to a string. Find out how to convert a float & double to QString. As well as convert a Qstring object into an integer/double/float values.

Converting Float/Double to QString:

To convert a float or double value to a QString you can use QString::number()

eg:

```
float fValue = 3.14159f;
double dValue = 2.71828;
QString floatString = QString::number(fValue);
QString doubleString = QString::number(dValue);
```

Converting QString to Integer/Double/Float:

To convert a QString to an integer, double, or float value; you can use toInt(), toDouble(), or toFloat()

eg:

```
QString intString = "42";
QString doubleString = "3.14159";
QString floatString = "2.71828";

int intValue = intString.toInt();
double doubleValue = doubleString.toDouble();
float floatValue = floatString.toFloat();
```

Jamie Goodrich-Maclean
1825924

A KeyPad Widget



In this example we will create a custom KeyPad Widget. This widget is basically a widget/control that can then be incorporated into another QWidget application in very much the same way as a QPushButton widget or a QLabel Widget. The KeyPad Widget must emit a signal whenever any of its buttons are pressed. This signal must pass the button's value as a parameter.

Since each button has its own clicked() signal, we can connect the clicked() signal of each button to a specific slot method. Each slot method can then emit the same custom signal but with a value that corresponds to the clicked button. This is a “many signals to single signal” mapping The code for this widget is provided below:

header file:

```
#ifndef KEYPAD_H
#define KEYPAD_H

#include <QWidget>
#include <QPushButton>
#include <QGridLayout>
#include <QDebug>
class keyPad : public QWidget
{
    Q_OBJECT

public:
    keyPad(QWidget *parent = 0);
    ~keyPad();
signals:
       void digitClicked(int digit);

private slots:
       void button0Clicked();
       void button1Clicked();
       void button2Clicked();
       void button3Clicked();
       void button4Clicked();
       void button5Clicked();
       void button6Clicked();
```

```
        void button7Clicked();
        void button8Clicked();
        void button9Clicked();

    private:
        void createLayout(void);
        QGridLayout *gridLayout;
        QPushButton *button[10];
};

#endif // KEYPAD_H
```

implementation

```
#include "keypad.h"
keyPad::keyPad(QWidget *parent)
    : QWidget(parent)
{
    createLayout();
    connect(button[0],SIGNAL(clicked()),this,SLOT(button0Clicked()));
    connect(button[1],SIGNAL(clicked()),this,SLOT(button1Clicked()));
    connect(button[2],SIGNAL(clicked()),this,SLOT(button2Clicked()));
    connect(button[3],SIGNAL(clicked()),this,SLOT(button3Clicked()));
    connect(button[4],SIGNAL(clicked()),this,SLOT(button4Clicked()));
    connect(button[5],SIGNAL(clicked()),this,SLOT(button5Clicked()));
    connect(button[6],SIGNAL(clicked()),this,SLOT(button6Clicked()));
    connect(button[7],SIGNAL(clicked()),this,SLOT(button7Clicked()));
    connect(button[8],SIGNAL(clicked()),this,SLOT(button8Clicked()));
    connect(button[9],SIGNAL(clicked()),this,SLOT(button9Clicked()));

}

keyPad::~keyPad()
{

}

void keyPad::createLayout(){

    gridLayout = new QGridLayout();

    for(int i=1; i<10 ; i++){
        button[i] = new QPushButton(QString::number(i));
        gridLayout->addWidget(button[i],((i-1)/3),((i-1)%3));
    }

    button[0] = new QPushButton(QString::number(0));
    gridLayout->addWidget(button[0],3,1);

    setLayout(gridLayout);

}

void keyPad::button0Clicked(){
```

```
    qDebug() << " Button 0 was clicked \n";
    emit digitClicked(0);
}

void keyPad::button1Clicked(){
    qDebug() << " Button 1 was clicked \n";
    emit digitClicked(1);
}

void keyPad::button2Clicked(){
    qDebug() << " Button 2 was clicked \n";
    emit digitClicked(2);
}

void keyPad::button3Clicked(){
    qDebug() << " Button 3 was clicked \n";
    emit digitClicked(3);
}

void keyPad::button4Clicked(){
    qDebug() << " Button 4 was clicked \n";
    emit digitClicked(4);
}

void keyPad::button5Clicked(){
    qDebug() << " Button 5 was clicked \n";
    emit digitClicked(5);
}

void keyPad::button6Clicked(){
    qDebug() << " Button 6 was clicked \n";
    emit digitClicked(6);
}

void keyPad::button7Clicked(){
    qDebug() << " Button 7 was clicked \n";
    emit digitClicked(7);
}

void keyPad::button8Clicked(){
    qDebug() << " Button 8 was clicked \n";
    emit digitClicked(8);
}

void keyPad::button9Clicked(){
    qDebug() << " Button 9 was clicked \n";
    emit digitClicked(9);
}
```

main.cpp

```
#include "keypad.h"
#include <QApplication>

int main(int argc, char *argv[])
```

```
{
    QApplication a(argc, argv);
    keyPad w;
    w.show();

    return a.exec();
}
```

Ex 11. The Keypad Widget's buttons are arranged in a 'grid' using the QGridLayout layout manager. Look up the QgridLayout in the Qt5 documentation and explain the operation of its "addWidget" method.

QGridLayout is a layout manager that can arrange widgets in a grid structure. addWidget is used to add widgets to the grid and customize.

example:

```
QGridLayout *gridLayout = new QGridLayout();

// add widgets to the grid layout at specific positions
QWidget *widget1 = new QPushButton("Button 1");
gridLayout->addWidget(widget1, 0, 0);

QWidget *widget2 = new QLabel("Label");
gridLayout->addWidget(widget2, 0, 1);

QWidget *widget3 = new QLineEdit();
gridLayout->addWidget(widget3, 1, 0);

// set the layout for the main widget
mainWidget->setLayout(gridLayout);
```

Here's an explanation of how the addWidget method works:

Note that in this example we map the clicked() signal of each button to an independent 'buttonNclicked()' slot. Each one of these slots emits the same custom signal 'digitClicked(int)' but with a different button value...i.e. the value of the button that was clicked. Notice how qDebug was used to test that the correct slot method was activated. Also a private method 'createLayout()' was used to draw out the widget. This method was then called in the constructor. The approach of drawing out the widget in a separate method, which is then called in the constructor is highly recommended as it improves the readability of the code.

This approach works but is kind of ugly. There's a lot of repetitive code that can make the example somewhat unwieldy. A better approach of mapping multiple signals to a single signal is to use the QsignalMapper approach.

```
#ifndef KEYPAD_H
#define KEYPAD_H

#include <QWidget>
#include <QPushButton>
#include <QGridLayout>
```

```
#include <QDebug>
#include <QSignalMapper>
class keyPad : public QWidget
{
    Q_OBJECT

public:
    keyPad(QWidget *parent = 0);
    ~keyPad();
signals:
       void digitClicked(int digit);

private:
       void createLayout(void);
       QGridLayout *gridLayout;
       QPushButton *button[10];
       QSignalMapper *signalMapper;
};

#endif // KEYPAD_H
```

implementation

```
#include "keypad.h"
keyPad::keyPad(QWidget *parent)
    : QWidget(parent)
{
    signalMapper= new QSignalMapper(this);
    connect(signalMapper, SIGNAL(mapped(int)), this, SIGNAL(digitClicked(int)));
    createLayout();

    for (int i = 0; i < 10; ++i) {
        signalMapper->setMapping(button[i], i);
        connect(button[i], SIGNAL(clicked()), signalMapper, SLOT(map()));

    }

}

keyPad::~keyPad()
{

}

void keyPad::createLayout(){

    gridLayout = new QGridLayout();

    for(int i=1; i<10 ; i++){
        button[i] = new QPushButton(QString::number(i));
        gridLayout->addWidget(button[i],((i-1)/3),((i-1)%3));
    }

    button[0] = new QPushButton(QString::number(0));
    gridLayout->addWidget(button[0],3,1);
```

```
    setLayout(gridLayout);

}
```

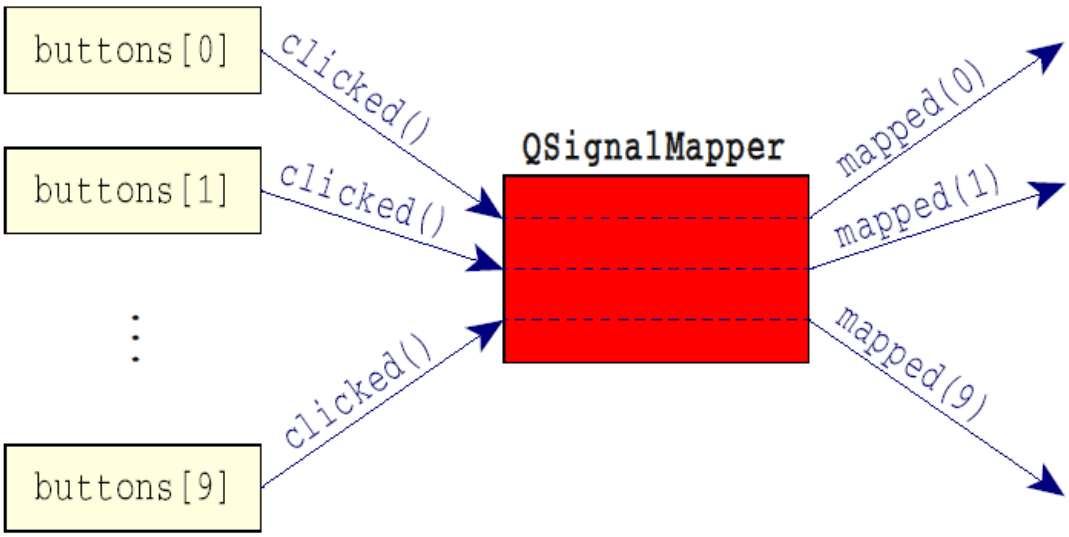
main.cpp

```
#include "keypad.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    keyPad w;
    w.show();

    return a.exec();
}
```

[1]Here we connect the multiple 'clicked()' signals to a QsignalMapper object. The call to `setMapping()` inside the for loop establishes a mapping between a button and an integer value; for example, `buttons[3]` is associated with the integer value 3.

When 

the `clicked()` signal of a button is emitted, QSignalMapper's `map()` slot is invoked. If a mapping exists for the object that emitted the signal, the `mapped(int)` signal is emitted with the integer value set using `setMapping()`. That signal is in turn connected to the `Keypad`'s `digitClicked(int)` signal (yes in Qt you can connect a signal to another.....). The `setMapping()` function exists in two versions: one that takes an `int` and one that takes a QString as the second argument. This makes it possible to associate an arbitrary string with a sender object, instead of an integer. When such a mapping exists, QSignalMapper emits a mapped(const QString &) signal.

Note the the QsignalMapper approach cannot be used with other data types. In that case consider using the sender or subclass methods discussed here https://doc.qt.io/archives/qq/qq10-signalmapper.html.

---

[1] http://doc.qt.digia.com/qq/qq10-signalmapper.html

Jamie Goodrich-Maclean
1825924

EX12 Add the keypad widget to a new PhoneDial form (shown in the figure below)!!! Create a new PhoneDial project (derived from QWidget) and add to it the keypad header and implementation files. The phoneDial form itself consists of a textfield (QLineEdit control), our custom keypad widget and a clear button all laid out in the form using a vertical layout manager.

Whenever a button on the keypad is pressed, its value will be inserted into the textfield. In otherwords, connect the keypad's 'digitClicked(int)' signal to a custom PhoneDial slot that will 'insert' the value of the clicked button into the text field. Also ensure that the 'clicked()' signal of the clear button is connected to another PhoneDial custom slot that 'clears' the textfield.

header phonedial:

```
#ifndef PHONEDIAL_H
#define PHONEDIAL_H

#include <QWidget>
#include <QLineEdit>
#include <QPushButton>
#include "keypad.h"

class PhoneDial : public QWidget
{
  Q_OBJECT

public:
  PhoneDial(QWidget *parent = nullptr);
  ~PhoneDial();

private slots:
  void handleDigitClicked(int digit);
  void handleClearClicked();

private:
  QLineEdit *lineEdit;
  keyPad *customKeypad;
  QPushButton *clearButton;
};

#endif // PHONEDIAL_H
```

phonedial cpp:

```
#include "phonedial.h"

PhoneDial::PhoneDial(QWidget *parent)
  : QWidget(parent)
{
  QVBoxLayout *layout = new QVBoxLayout();

  lineEdit = new QLineEdit();
  layout->addWidget(lineEdit);
```

```
  customKeypad = new keyPad();
  layout->addWidget(customKeypad);

  clearButton = new QPushButton("Clear");
  layout->addWidget(clearButton);

  connect(customKeypad, SIGNAL(digitClicked(int)), this, SLOT(handleDigitClicked(int)));
  connect(clearButton, SIGNAL(clicked()), this, SLOT(handleClearClicked()));

  setLayout(layout);
}

PhoneDial::~PhoneDial()
{
}

void PhoneDial::handleDigitClicked(int digit)
{
  // Append the digit to the text field
  lineEdit->setText(lineEdit->text() + QString::number(digit));
}

void PhoneDial::handleClearClicked()
{
  // Clear the text field
  lineEdit->clear();
}
```



Ex13: Re-Implement the QKeyPress event for the PhoneDial application. To account for the situation when the user chooses to type in the phone number rather than click the buttons.

header:

```
#ifndef PHONEDIAL_H
#define PHONEDIAL_H

#include <QWidget>
#include <QLineEdit>
#include <QPushButton>
#include "keypad.h"

class PhoneDial : public QWidget
{
  Q_OBJECT

public:
  PhoneDial(QWidget *parent = nullptr);
  ~PhoneDial();

private slots:
  void handleDigitClicked(int digit);
  void handleClearClicked();

protected:
  void keyPressEvent(QKeyEvent *event); // Re-implement keyPressEvent

private:
  QLineEdit *lineEdit;
  keyPad *customKeypad;
  QPushButton *clearButton;
};

#endif // PHONEDIAL_H
```

implementation cpp:

```
#include "phonedial.h"
#include <QKeyEvent>

PhoneDial::PhoneDial(QWidget *parent)
  : QWidget(parent)
{
  QVBoxLayout *layout = new QVBoxLayout();

  lineEdit = new QLineEdit();
  layout->addWidget(lineEdit);

  customKeypad = new keyPad();
  layout->addWidget(customKeypad);

  clearButton = new QPushButton("Clear");
  layout->addWidget(clearButton);
```

```
    connect(customKeypad, SIGNAL(digitClicked(int)), this, SLOT(handleDigitClicked(int)));
    connect(clearButton, SIGNAL(clicked()), this, SLOT(handleClearClicked()));

    setLayout(layout);
}

PhoneDial::~PhoneDial()
{
}

void PhoneDial::handleDigitClicked(int digit)
{
    // Append the digit to the text field
    lineEdit->setText(lineEdit->text() + QString::number(digit));
}

void PhoneDial::handleClearClicked()
{
    // Clear the text field
    lineEdit->clear();
}

void PhoneDial::keyPressEvent(QKeyEvent *event)
{
    // Check if the key event corresponds to a numeric key (0-9)
    if (event->key() >= Qt::Key_0 && event->key() <= Qt::Key_9) {
        // Get the numeric value from the key event
        int digit = event->key() - Qt::Key_0;

        // Append the digit to the text field
        lineEdit->setText(lineEdit->text() + QString::number(digit));
    }
    else if (event->key() == Qt::Key_Backspace) {
        // Handle backspace key to remove the last character
        QString currentText = lineEdit->text();
        if (!currentText.isEmpty()) {
            currentText.chop(1); // Remove the last character
            lineEdit->setText(currentText);
        }
    }
    else {
        // Call the base class for other key events
        QWidget::keyPressEvent(event);
    }
}
```